Online Store

Intro to DataBase Project

Explanation

- Project Components:
  - Relational Schema/EER
  - Code
- Code Details:
  - MySQL (DDL, DML, DML 7 queries)
  - PHP/HTML/CSS/JavaScript (GUI)
- Relational Schema Details:
  - EER
  - Mapping
  - Data Dictionary (Meta Data)
- Purpose: All components combine to create an Online Store Software.

Data Dictionary

The process started with the database requirements collection, then it was translated into a data dictionary which offers: Data Type, Key Type, Description. For every Attribute in every Table.

Data Dictionary Sample:-

User_phone

Attribute	Data Type	Key Type	Description
user_id	INT	FK	References User.user_id
phone	VARCHAR(20)	-	User phone number

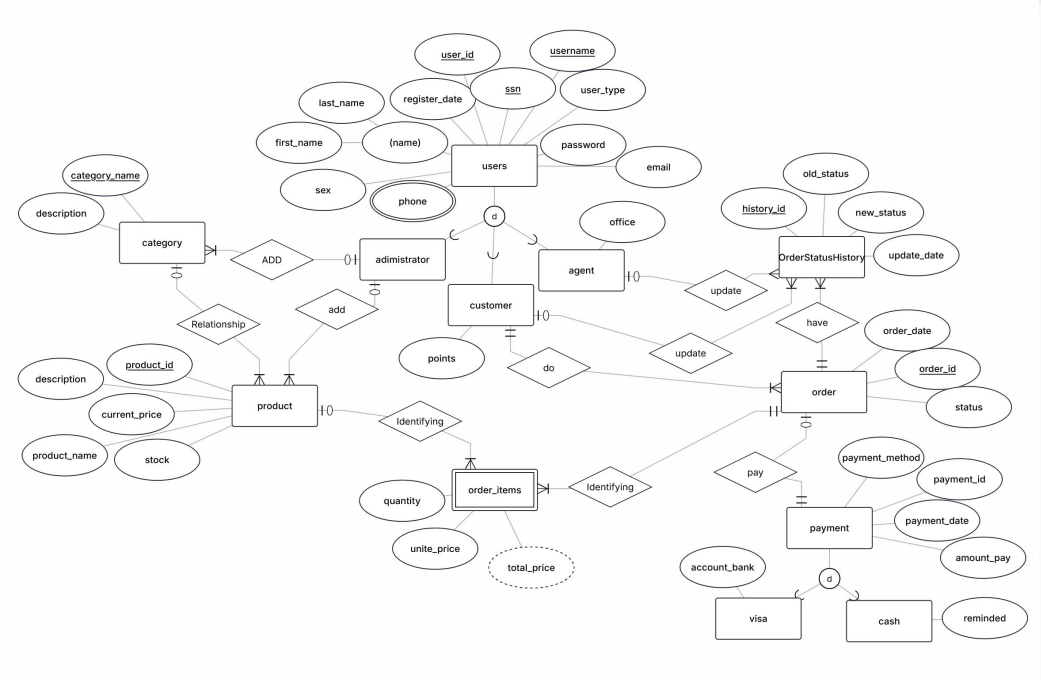
Agent

Attribute	Data Type	Key Type	Description
agent_id	INT	PK	Unique identifier for agent
office	VARCHAR(50)	-	Agent office

EER

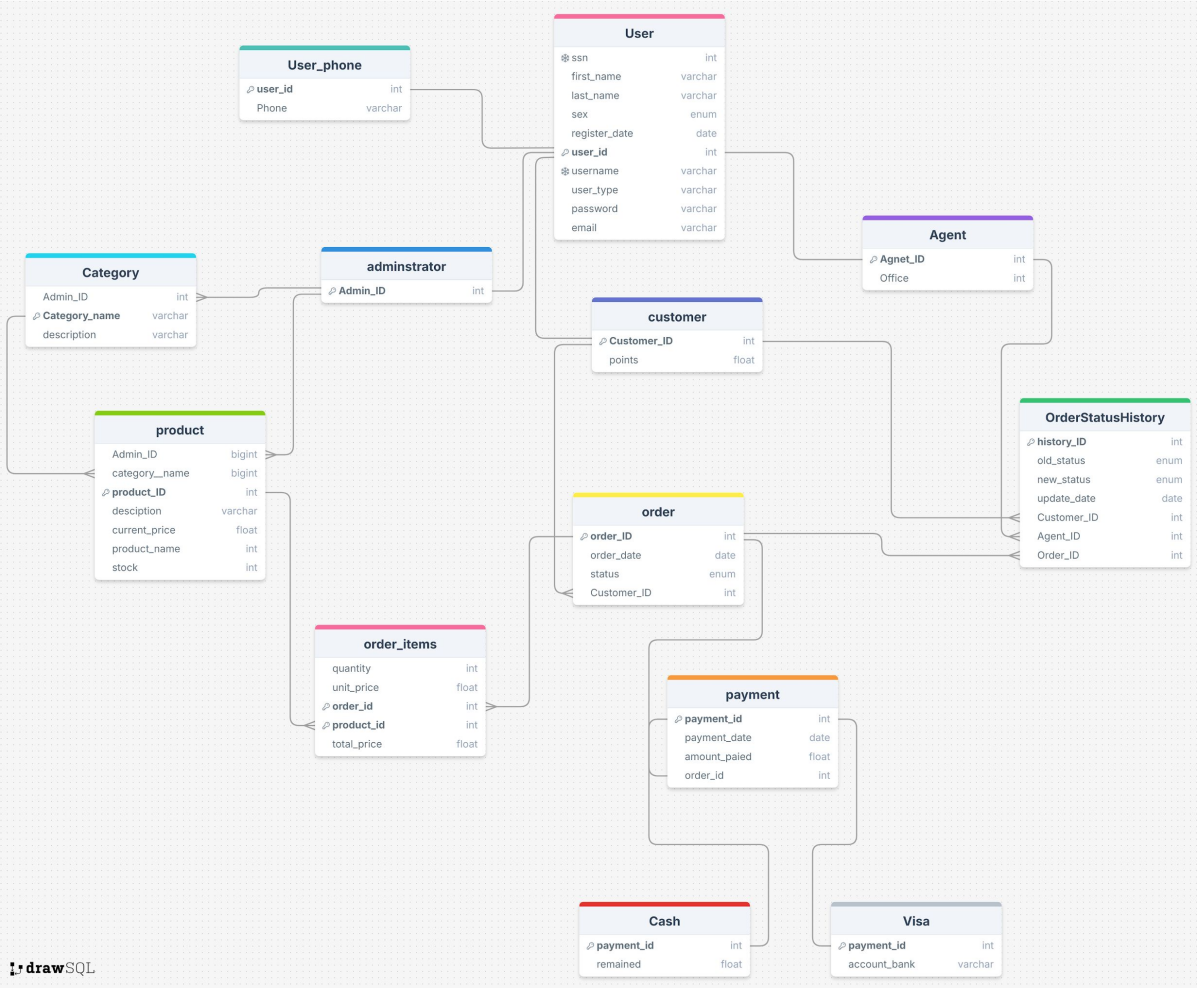
The Enhanced Entity-Relationship (EER) model builds upon existing [database requirements and the data dictionary](). It is used to:

- Assign attributes to entities.
- Define relationships between entities, including one-to-one (1:1), one-to-many (1:M), many-to-one (M:1), and many-to-many (M:M).
- Introduce junction entities to resolve many-to-many (M:M) relationships, as these relationships are not directly processable by Database Management Systems (DBMS).



Mapping

- **Entities to Tables:** Each strong entity and weak entity in the [EER model]() is mapped directly to its own relational table.
- **Attributes to Columns:** All simple attributes become columns in the corresponding table. Composite attributes are broken down into their simple components.
- **Primary Keys:** The primary key of the entity is chosen as the primary key for the table. For a weak entity, the primary key is a combination of the partial key and the primary key of its identifying entity.
- **Relationships to Foreign Keys/Tables:** One-to-One (1:1) and One-to-Many (1:M) relationships are typically mapped using foreign keys. Many-to-Many (M:M) relationships are resolved by creating a new **junction table** whose primary key is a composite of the primary keys of the participating entities.
- **Generalization/Specialization:** Subclasses can be mapped by creating separate tables for each subclass and linking them to the superclass table via foreign keys, or by combining all attributes into a single table with a type discriminator column.
- **All of this was then taken to drawSQL to illustrate it digitally**



Data Definition Language (DDL):

- The DDL was written in the query box of PHPMyAdmin.
- The data dictionary, mapping, and EER diagram were used as references.
- Using DDL the DataBase and the 13 Tables were created with all their attributes and types and keys

DDL Sample:-

```
create database online_store;

use online_store;

create table Users(
      User_ID bigint primary key auto_increment,
      User Name varchar(256) unique not null,
      SSN int unique not null,
      First_Name varchar(100) not null,
      Last Name varchar(100) not null,
      Sex enum('Male','Female','Other'),
      Register Date date,
      password varchar(256) not null,
      Email varchar(256) not null
);


alter table Users
modify column SSN varchar(14) not null unique;

SHOW INDEX FROM Users;

alter table Users
drop index SSN_2;
```

Data Manipulation Language (DML):

- DML is used to manage and manipulate data in tables.
- The `INSERT` command populates tables with initial records.
- The `SELECT` command retrieves data for review and reporting.
- Complex queries use `INNER JOIN` to link data across tables.
- DML confirms the database is functional for application use.

DML Sample:-

```
-- Admin --

insert into Administrators (Admin_ID)
values (1);

select * from Administrators;

select *
      from Administrators ad
      inner join Users u
      on Ad.Admin_ID = U.User_ID;


-- agent--

insert into Agents (Agent_ID , Office)
values
      (3 , 1),
    (5 , 2);

select * from Agents;

select *
      from Agents ag
      inner join Users u
      on ag.Agent_ID = u.User_ID;
```

Seven Queries

"The following seven queries demonstrate the functional capability of the Online Store database and validate the relational schema. Each query is designed to retrieve specific, useful business intelligence by joining data across multiple tables."

- **Query 1:** Links `Customers` to `Users` to show customer details and their accumulated loyalty `points`.
- **Query 2:** Links `Products` to `Categories` to show which categories popular products fall into and their current pricing.
- **Query 3:** Links `Products` to `Administrators` and then to `Users` to show which administrator added or manages a specific product.
- **Query 4:** Links `Orders` to `Customers` and `Users` to display an order's status and the customer who placed it.
- **Query 5:** Links `Order_Items` to `Products` to show the details (name and price) of items within a specific order.
- **Query 6:** Links `OrderStatusHistory` to `Customers` and `Users` to track the full status evolution of an order over time.
- **Query 7:** A complex query that links `Orders`, `Customers`, `Order_Items`, and `Products` to provide a consolidated view of who bought what and when.

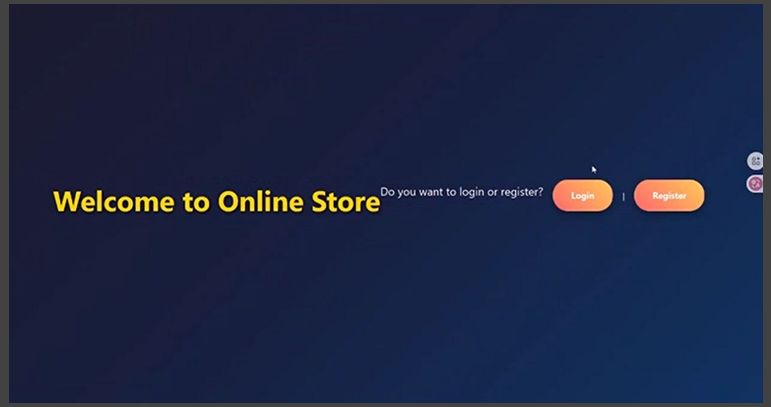
```
SELECT Users.user_id, Users.first_name, Users.last_name, Users.email, Customers.points
FROM Customers
JOIN Users ON Users.user_id = Customers.customer_id;

SELECT Products.product_name, Products.current_price, Categories.category_name
FROM Products
JOIN Categories ON Categories.category_name = Products.category_name;

SELECT Products.product_name, Users.first_name, Users.last_name
FROM Products
JOIN Administrators ON Administrators.admin id = Products.admin_id
JOIN Users ON Users.user_id = Administrators.admin_id;

SELECT Orders.order_id, Orders.status, Users.first_name, Users.last_name
FROM Orders
JOIN Customers ON Customers.customer id = Orders.customer_id
JOIN Users ON Users.user_id = Customers.customer_id;

SELECT Order Items.order_id, Products.product_name, Products.current_price
FROM Order Items
JOIN Products ON Products.product_id = Order_Items.product_id;

SELECT OrderStatusHistory.history id, OrderStatusHistory.old_status, OrderStatusHistory.new_status,
       Users.first name, Users.last_name
FROM OrderStatusHistory
JOIN Customers ON Customers.customer id = OrderStatusHistory.customer_id
JOIN Users ON Users.user_id = Customers.customer_id;

SELECT Orders.order_id, Orders.order_date, Users.first_name, Products.product_name
FROM Orders
JOIN Customers ON Customers.customer id = Orders.customer_id
JOIN Users ON Users.user id = Customers.customer id
JOIN Order Items ON Order Items.order id = Orders.order id
JOIN Products ON Products.product_id = Order_Items.product_id;
```

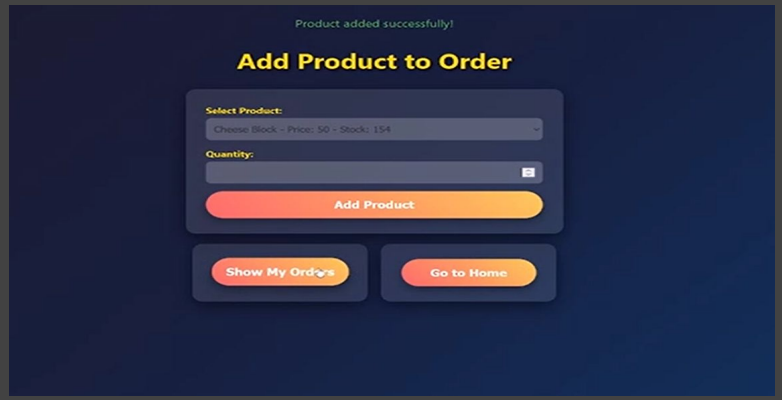
The Gui is a bonus grade on the project

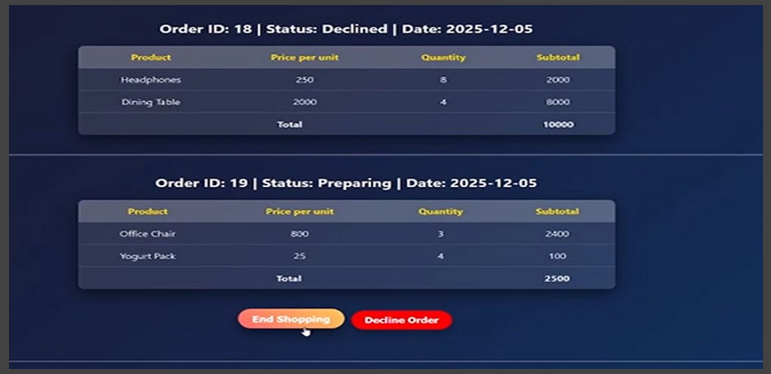
It is made using PHP mainly

GUI

A GUI Sample


Welcome to Online Store
Do you want to login or register?
Login
Register


Product added successfully!
Add Product to Order
Select Product:
Cheese Block - Price: 50 - Stock: 154
Quantity:
Add Product
Show My Orders
Go to Home


Order ID: 18 | Status: Declined | Date: 2025-12-05
Product | Price per unit | Quantity | Subtotal
Headphones | 250 | 8 | 2000
Dining Table | 2000 | 4 | 8000
Total | 10000
Order ID: 19 | Status: Preparing | Date: 2025-12-05
Product | Price per unit | Quantity | Subtotal
Office Chair | 800 | 3 | 2400
Yogurt Pack | 25 | 4 | 100
Total | 2500
End Shopping
Decline Order

Update Your Information
Username:
ebrahim0011
SSN:
4
First Name:
ebrahim
Last Name:
mohamed
Sex:
Male
Email:
ebrahim@gmail.com
Password:
123456

The project provided an exceptional exploration of every facet of MySQL, proving to be a highly valuable and insightful experience for all participants.

~Youssef Zaki

Thanks!

Made By:-

Youssef Walid Elsayed Zaki
2401245448
Ahmed Sameh Eid Salama
2401247903
Ahmed Sarhan Mohamed
2401241995
Youssef Wael Abd Elsalam
2401241397
Jack Gerges
2401244171
Fady Nasser Ramla
2401244980